

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о программном проекте на тему:

Колоночный движок с нуля

(промежуточный, этап 1)

Выполнил студент:

группы #БПМИ243, 2 курса

Бужин Игорь Тимурович

Принял руководитель проекта:

Перов Герман Сергеевич

Научный сотрудник

Факультет компьютерных наук НИУ ВШЭ

Содержание

Аннотация	3
1 Введение	4
1.1 Описание предметной области	4
1.2 Постановка задачи	4
1.3 Структура работы	4
2 Обзор литературы	5
2.1 ClickBench	5
2.2 Колоночные движки	5
2.3 Основные понятия и определения	5
3 Описание предлагаемого метода	7
3.1 Структура input output логики	7
3.2 Колоночный формат bzn и логика типов в классе Column	8
3.3 Логика исполнения запросов	11
3.3.1 Агрегации	11
3.3.2 Выражения над колонками	12
3.3.3 Операторы	14
4 Эксперименты	16
4.1 Базовая работа конвертации	16
4.2 Время чтения в зависимости от количества колонок	16
4.3 Сравнение с DuckDB	17
5 План совершенствования проекта	19
6 Заключение	19
Список литературы	20

Аннотация

Цель проекта создать колоночный формат и реализовать колоночный движок, который будет успешно проходить бенчмарк ClickBench.

Ключевые слова

Колоночный движок, колоночный формат, базы данных, ClickBench, pull, columnar engine.

1 Введение

1.1 Описание предметной области

СУБД (Системы Управления Базами Данных) являются неотъемлемой частью любого крупного сервиса. Они позволяют эффективно хранить и обрабатывать огромные объемы данных, отвечая на SQL запросы агрегаций, группировок, фильтров и других операций.

В частности, колоночные движки являются популярным решением для исполнения SQL запросов к таблицам, которые не требуют функционала добавления/удаления строк. У них есть свой колоночный формат (данные лежат столбцами, а не строками), что минимизирует количество *cache miss*'ов (промахов по кэшу процессора, что влечет загрузку новых кэш-линий в процессор), на запросах, использующих малую часть колонок и чтений с диска при ответе на запрос. Благодаря этому колоночные движки могут быть в разы эффективнее, чем строковые движки при ответе на запросы, которые обращаются лишь к паре столбцов.

Для оценки производительности движков используем ClickBench. Этот способ позволяет оценить эффективность аналитических движков, бенчмарк прогоняет запросы на таблице, которая содержит порядка 100 столбцов и около 90 миллионов строк, при этом все данные анонимизированы и основаны на работе популярного сервиса, что делает условия максимально приближенными к реальным.

1.2 Постановка задачи

Переложить данные, в свой колоночный формат и реализовать колоночный движок, работающий с новым форматом, который сможет успешно проходить бенчмарк ClickBench.

1.3 Структура работы

Колоночный формат: класс для чтения и записи csv файла, а также разработан колоночный формат и класс для чтения и записи bzip формата.

Легко масштабируемый движок, построенный на pull архитектуре. Благодаря простым интерфейсам операторов, выражений и агрегаций движок является очень гибким и добавление нового функционала не составит труда.

Исходный код проекта[5].

2 Обзор литературы

2.1 ClickBench

ClickBench [2] – это инструмент, который позволяет протестировать производительность БД на базовой нагрузке сфер бизнес аналитики и дашбордов реального времени. Универсальный бенчмарк, который используя самые базовые операции в запросах оценивает эффективность СУБД.

Данные представляют собой фактическую запись трафика из одной из крупнейших в мире платформ в сфере веб-аналитики. Они анонимны и лежат в виде одной плоской таблицы на 99 997 497 строк и 105 столбцов. Всего в бенчмарке 43 SQL запроса, они представляют собой: базовые агрегации, группировки, сортировки+limit, а также тяжелые запросы. При прогоне запросов запрещено кэширование, каждый запрос прогоняется три раза.

Целью является прохождение бенчмарка ClickBench, лидеры по производительности расположены на сайте бенчмарка [2]. Результаты запуском на сайте представлены на различных машинах, как с cold (запуск на свежей машине), так и warm (запуск с прогретыми под нашу задачу кешами и процессором) режимами. Выберем тройку лидеров (все они являются колоночными). ClickHouse [3], Parquet [1] и DuckDB [4]. Выделим особенности каждого из движков.

2.2 Колоночные движки

ClickHouse [3] использует собственные форматы хранения (семейство движков MergeTree), сжатие и векторные инструкции процессора для обработки данных.

DuckDB [4] — это встраиваемая (in-process) база данных. У неё нет сервера, она работает внутри процесса и имеет хорошую совместимость, что и есть ее главные особенности.

Parquet [1] — это колоночный формат хранения данных, который содержит метаданные (статистику min/max), что позволяет движкам читать только нужные части файла.

У всех трёх движков есть свои особенности, которые позволяют им быть производительными.

2.3 Основные понятия и определения

Введем определение batch'а. Скажем, что количество колонок в нашей таблице это m . batch — это часть таблицы, которая помещается в оперативную память. Данные столбцов лежат рядом друг с другом. На это можно смотреть так: возьмем k подряд идущих строк

таблицы, транспонируем их и наш batch готов. То есть вся исходная таблица представима в виде $batch_1 \sqcup batch_2 \sqcup \dots \sqcup batch_k$, где k — количество batch'ей, зависящее от количества строк в таблице и размера batch'a (на данный момент размер определяется как количество, хранящихся в batch'e). Для наглядной картинки можно обратиться к формату файла движка Apache Parquet [1]. У них batch — это row group (chunk).

Стоит ознакомиться с SQL синтаксисом, запросы исполняемые движком лежат в репозитории проекта [5], файл queries.sql, осознание того что делают эти запросы будет очень хорошим пререквизитом для полного понимания структуры работы.

Pull-based подход — это архитектура, в которой каждому оператору не важно какой оператор на входе и какой оператор на выходе. Все что есть у операторов для взаимодействия друг с другом это метод `Next`, который прокидывает батчи между операторами, пока есть что прокидывать. В качестве примера рассмотрим задачу: нужно найти количество строк в таблице с непустой строкой в колонке j . В таком случае это последовательность операторов `scan` $\rightarrow$ `filter` $\rightarrow$ `aggregate`. Для результата нужно позвать `Next` от `aggregate`, он в свою очередь будет звать `Next` от `filter` пока не закончатся данные, `filter` зовет `Next` от `scan`, `scan` дает батч, `filter` оставляет только строки с непустой строкой в колонке j , дает батч `aggregate`'у, тот в свою очередь накапливает количество строк в батчах, которые дает `filter` и возвращает ответ. Похожий метод используется в статье [6], в которой также есть pull модель, но более сильная в плане параллелизации.

Определим выражение. Это какая-то последовательность действий с колонками, выстраивающихся в Abstract Syntax Tree. Для осознания того, что такое AST, можно посмотреть на построение дерева для польской записи математических выражений, или разобраться в устройстве LISP языка. Листовыми выражениями будем называть ссылку на колонку или литерал, подробнее в описании структуры.

План запросов — это композиция операторов и выражений над колонками, которая нацелена на исполнение конкретного sql запроса.

Парсер — это программа, которая занимается построением плана запросов по sql запросам.

3 Описание предлагаемого метода

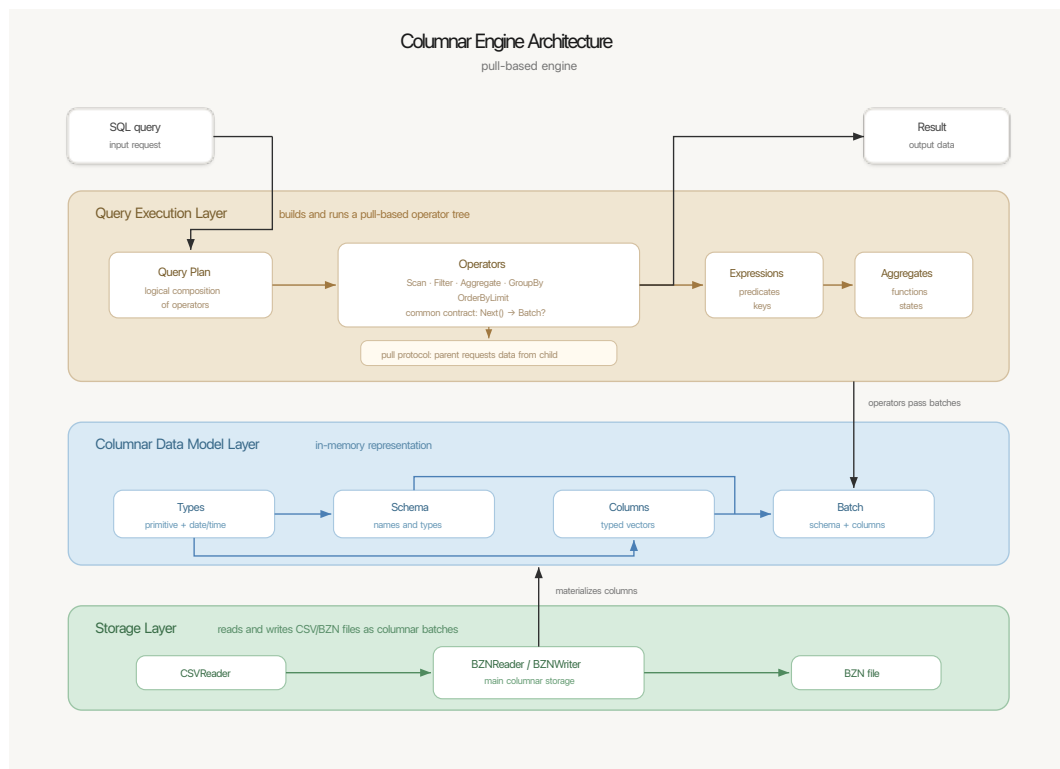


Рис. 3.1: Архитектура проекта Columnar Engine.

3.1 Структура input output логики

- `rwconsts.h` - файл, содержащий константы для размера батча, разделителей строк в метаданных.
- `types.h` - реализация логики типов, конвертация между типами, `src` тип по `enum` типу.
- `datatype.h` - логика работы с датами. Описано в разделе о `bzn` формате.
- `class Schema` — содержит информацию о названиях и типах столбцов таблицы.
- `class Column` — содержит данные одного столбца таблицы, представляет из себя `std::variant<std::vector<type_1>, ...>` имеет метод `TranslateTo`, который переводит колонку из одного типа в другой.
- `class Batch` — batch, определенный ранее, хранит данные как `std::vector<Column>` и держит рядом класс `Schema`, это сделано для того чтобы в батч можно было подставить другую схему, что вызовет трансформацию столбцов (`TranslateTo`) в нужные типы.
- `class CSV-Reader` — читает csv файл согласно RFC-4180 [7] и возвращает batch строкового типа.

- `class CSV-Writer` — принимает `batch`, дает ему новую схему в которой типы это строка, данные в батче переводятся в строки, записываем в csv файл согласно RFC-4180 [7].
- `class BZN-Reader` — читает колоночный формат и возвращает `batch`. В конструкторе принимает множество колонок, которые нужно читать, при передаче пустого списка читает все колонки.
- `class BZN-Writer` — принимает `batch` и записывает в колоночный формат (при инициализации создает массив с разметкой батчей, который заполняется по ходу записи).

Для чтения пользуемся `fstream` так как это достаточно простой и понятный метод. В случае csv читаем побайтово, в случае с bzn читаем все кроме строк сразу блоком, строки по специфике хранения читаем побайтово. Сделать `mmap` файла потребовало бы более тяжелой логики и могло дать хороший рост производительности в io части, но такой эксперимент потребовал бы много времени.

3.2 Колоночный формат bzn и логика типов в классе `Column`

Поддерживаются следующие типы данных: `bool`, `int16_t`, `int32_t`, `int64_t`, `double`, `long double`, `string`, `date`, `timestamp`.

- `bool`: Не храним явно на диске, этот тип колонки используется только на этапе вычисления выражений с колонками.
- `int16_t`, `int32_t`, `int64_t`:
 - `csv → bzn : std::from_chars` от строкового представления числа в `int_(16|32|64)_t` так как работает быстрее, чем `std::stoll`, кладем побайтово `sizeof` байт в little-endian формате.
 - `bzn → csv : std::to_string` целого числа.
- `double`:
 - `csv → bzn : std::from_chars` от строкового представления числа в `double`, далее `memcpy sizeof(double)` байт.
 - `bzn → csv : std::to_string` вещественного числа.
- `long double`:

- `csv` → `bzn : std::stold` так как `std::from_chars` в силу зависимости от системы не поддерживается, кладем в `long double`, далее `memcpy sizeof` байт.
- `bzn` → `csv : std::to_string` вещественного числа.

- **string:**

- `csv` → `bzn` : Записывается как последовательность байт(символов), описывающая нашу строку, после чего идет `'\x1F'` — специальный символ, демонстрирующий конец строки.
- `bzn` → `csv` : читаем до `'\x1F'` и записываем строку в `csv`.

- **date:** Храним количество дней(в нашей системе координат, особенности которой в следующем абзаце) с 1970.01.01 в `int32_t`.

В году 512 дней, в месяце 32 дня, потому что в нашей задаче для дат требуется только сравнение и транслирование `yyyy-mm-dd` ↔ `int32_t`, очевидно, что порядок в таком случае сохраняется, но теряются операции сложения и вычитания, что можно компенсировать обычными методом конвертирования.

Этот подход позволил тратить меньше чем встроенные парсеры, и чем свой ранее написанный парсер, который хранил префиксные суммы дней в зависимости от года от 1970 года, потому что теперь все деления и нетривиальные операции заменяются быстрыми операциями со степенями двойки.

- `csv` → `bzn : std::from_chars` от `yyyy, mm, dd`, далее `yyyy * 512 + mm * 32 + dd` → `int32_t`.
- `bzn` → `csv` : обратная процедура с `std::to_string`.

- **timestamp:** Храним количество секунд с 1970.01.01-00:00:00 в `int64_t`.

- `csv` → `bzn` : получаем дни `dd` от `data`, далее `std::from_chars` от `hh, mm, ss`, далее `dd * 24 * 3600 + hh * 3600 + mm * 60 + ss` → `int64_t`.
- `bzn` → `csv` : обратная процедура с `std::to_string`.

batch в `bzn` формате лежит как:

$$[\text{метаданные}] [\text{данные}] \quad (1)$$



Рис. 3.2: Схема хранения файла.

Метаданные хранятся как подряд идущие `int64_t`, которые описывают позиции в файле, с которых начинается каждая колонка $poscolumn_1, poscolumn_2, \dots, poscolumn_m$. После мета-информации лежат сами данные колонок, с позиции $poscolumn_i$ в файле лежит i -ая колонка (о том как хранятся различные типы уже говорили). Далее поговорим о метаданных всего файла. Из которых будет очевидно количество колонок в таблице, а значит и задача чтения батча очевидна.

Формат `bzn` выглядит следующим образом, $batch_i$ хранится как сказано выше:

$$[\text{размер метаданных}] [batch_1] [batch_2] \dots [batch_k] [\text{метаданные}] \quad (2)$$

Метаданные в конце файла хранятся как:

$$[\text{имена столбцов}] [\text{типы столбцов}] [\text{разметка батчей}] \quad (3)$$

Где имена столбцов — это последовательность строк, разделенных символом `'\x1F'`, после которых идет символ `'\x1E'`, говорящий об окончании этой секции метаданных. Типы столбцов хранятся аналогично их строковыми представлениями. После них также идет сим-

вол `'\x1E'` и начинается разметка батчей, которая представляет собой последовательность `int64_t`, каждый из которых говорит о позиции в файле, с которой начинается соответствующий батч.

3.3 Логика исполнения запросов

Логика исполнения запросов нацелена на простое построение плана. Для пользователя созданы базовые интерфейсы агрегаций, выражений над колонками и операторов, которые позволяют без проблем добавить новый функционал и построить план исполнения запроса. Для добавления нового функционала, его нужно унаследовать от базового класса с интерфейсами и написать собственную реализацию нового метода.

3.3.1 Агрегации

Базовый интерфейс `IAggregateFunc`, агрегации, унаследованные от этого класса, должны переопределить методы:

- `std::shared_ptr<IAggregateState> CreateState()` : создание пустого состояния агрегации.
- `void Update(std::shared_ptr<IAggregateState>, const Column&)` : обновление состояния по очередному батчу значений.
- `Column Finalize(std::shared_ptr<IAggregateState>)` : получение итоговой колонки из одного элемента.

Состояние агрегации отделено от самой функции агрегации. Это позволяет одному объекту агрегации создавать разные независимые состояния для разных групп в операторе `GroupByOperator`: для каждого ключа группировки хранится свой `IAggregateState`, а при поступлении нового батча соответствующее состояние пересчитывается методом `Update`. После обработки всех батчей метод `Finalize` превращает накопленное состояние в одноэлементную колонку результата. Для создания state нужно определить деструктор и поля которые будут в нем лежать.

Поддерживаемые агрегации:

- `CountFunc` : считает количество строк во входной колонке. В состоянии `CountState` хранится одно поле `size_t res` — текущее количество элементов. При пересчете состояние увеличивается на `col.GetSize()`. В `Finalize` возвращается колонка типа `int32_t` из одного элемента, равного накопленному счетчику.

- **SumFunc** : считает сумму значений числовой колонки. В состоянии **SumState<T>** хранится поле **T res**, где **T** – тип колонки. При пересчете агрегация проходит по всем значениям входной колонки и прибавляет их к **res**. В **Finalize** для целых типов возвращается **int64_t**, для вещественных — **long double**.
- **AvgFunc** : считает среднее значение числовой колонки. В состоянии **AvgState** хранятся два поля: **size_t cnt** — количество уже обработанных элементов и **long double sum** — сумма обработанных значений. При пересчете **cnt** увеличивается на размер входной колонки, а к **sum** прибавляется сумма всех элементов батча. В **Finalize** возвращается колонка типа **long double** со значением **sum / cnt**.
- **CountDistinctFunc** : считает количество различных значений. В состоянии **CountDistinctState** хранится **std::set<T> res**, где **T** соответствует типу входной колонки. При пересчете каждый элемент входной колонки вставляется в множество. В **Finalize** возвращается колонка типа **int32_t** со значением **res.size()**. В будущем можно перейти на **std::unordered_set**.
- **MinFunc** : считает минимум по входной колонке. В состоянии **MinState<T>** хранится текущее минимальное значение **T res**. Для числовых типов начальное значение равно **std::numeric_limits<T>::max()**, для строк — строка из байтов **'\xFF'**, чтобы первое реальное значение могло уменьшить состояние. При пересчете для каждого элемента выполняется **res = std::min(res, value)**. В **Finalize** возвращается одноэлементная колонка того же типа, что и входная. Для **bool** минимум не поддерживается.
- **MaxFunc** : считает максимум по входной колонке. В состоянии **MaxState<T>** хранится текущее максимальное значение **T res**. Для числовых типов начальное значение равно **std::numeric_limits<T>::lowest()**. При пересчете для каждого элемента выполняется **res = std::max(res, value)**. В **Finalize** возвращается одноэлементная колонка того же типа, что и входная. Для **bool** максимум не поддерживается.

3.3.2 Выражения над колонками

Базовый интерфейс **IExpression**, выражения, унаследованные от этого класса должны переопределить методы:

- **Column Evaluate(const Batch&)** : вычисление выражения.
- **std::string GetName() const** : вернуть имя колонки.

Каждое выражение должно иметь в конструкторе имя колонки, которое будет использоваться в плане исполнения запроса. В случае неуказания имени оно будет пустым и потеряется возможность обращения к колонке между операторами.

Поддерживаемые выражения:

- **ColumnRef** : принимает в конструкторе строку(название колонки которую нужно взять), забирает колонку из батча, листовое выражение.
- **Literal<T>** : принимает в конструкторе тип колонки, базовое значение типа **T** и имя. Строит новую колонку, заполненную базовым значением, листовое выражение.
- **BinaryCmp** : принимает в конструкторе левого и правого сына, **enum** класс с типами сравнений, поддерживается $<, \leq, =, >, \geq, \neq$. Вызывает **Evaluate** от левого и правого сына и проводит сравнения. Возвращает колонку из **bool** элементов.
- **BinaryFunc** : принимает в конструкторе левого и правого сына, **enum** класс с типом операции, поддерживается $+, -, and, or$. Вызывает **Evaluate** от левого и правого сына и применяет бинарную операцию. Возвращает колонку, являющуюся результатом поэлементного применения бинарной функции к двум колонкам.
- **Like** : принимает в конструкторе колонку строкового типа, паттерн, вхождение которого ищем в колонке, флаг, отвечающий за инвертирование ответа и имя. Строит новую колонку типа **bool** который отвечает за вхождение подстроки паттерн в i -ой позиции колонки (или не вхождение в зависимости от **bool**). Поиск подстроки осуществляется через префикс функцию и работает за $O(n)$ памяти, где n - размер паттерна и за $O(n + l)$ операций, где l суммарная длина строк в колонке. Идея строится на базовом алгоритме поиска вхождений строки t в строку s с помощью префикс функции, где мы склеивали t и s через сепаратор и явно строили префикс функцию, только мы ее усовершенствовали построив префикс функцию только для t .
- **ExtractFromTime** : принимает в конструкторе дочернее выражение, которое должно быть типа **timestamp**, и имя колонки. Выдает колонку, состоящую из минут.
- **TruncateTime** : принимает в конструкторе дочернее выражение, которое должно возвращать колонку типа **timestamp**, **enum class** **Trunc**, который говорит до каких единиц округлять, и имя колонки. Выдает колонку, округленную вниз до минут, часов, дней, месяцев, лет в зависимости от переданного **Trunc**.

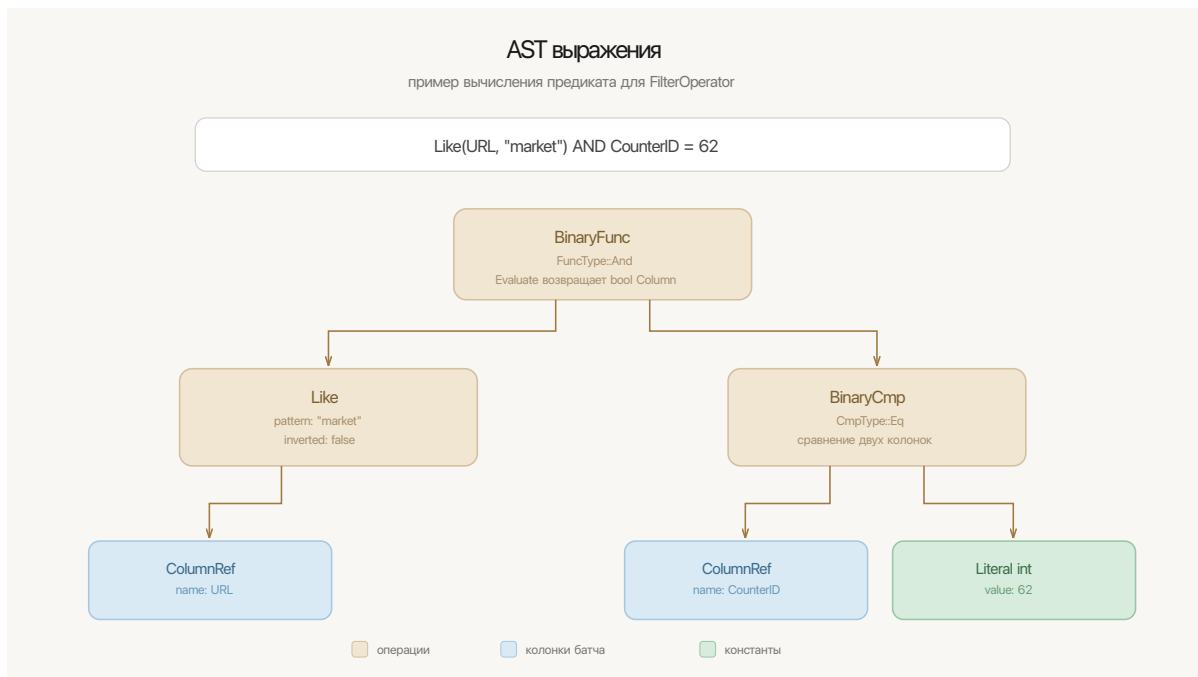


Рис. 3.3: Пример AST для выражения `Like(URL, "market") AND CounterID = 62`.

3.3.3 Операторы

Базовый интерфейс `IOperator`. Любой оператор должен переопределить методы:

- `std::optional<Batch> Next()` : получить следующий батч.

Метод `Next` реализует pull-based модель исполнения: родительский оператор для получения результата запрашивает данные у своего дочернего оператора. Если данные еще есть, `Next` возвращает `Batch`. Если входной поток закончился, возвращается `std::nullopt`. За счет этого весь план запроса можно собрать как цепочку объектов `IOperator`, где каждый оператор хранит указатель на `child` и вызывает его `Next` при необходимости.

Реализованы следующие операторы:

- **ScanOperator** : листовой оператор, читает из `BZNReader` только указанные колонки и возвращает батчи из файла.
- **FilterOperator** : хранит дочерний оператор и предикат `IEExpression`. На каждом батче вычисляет предикат, получает булеву маску и оставляет только подходящие строки.
- **AggregateOperator** : хранит список агрегаций и выражений, вычисляет выражения над входными батчами и накапливает агрегатные состояния без группировки.
- **GroupByOperator** : расширяет агрегацию группировкой по ключевым выражениям. Для каждого набора ключей хранит отдельные состояния агрегаций. На данный момент

группировка производится с помощью `map<string, state>`. Предположим, нужно сделать группировку по колонкам а и б с типами строка и число. В таком случае мы переводим все колонки в строковый тип, склеиваем их через разделитель, делаем группировку, на выходе из оператора разбиваем строки по разделителю обратно (при этом ключи становятся строками). Этот подход самый понятный и простой, за счет этого он является не самым оптимальным и проигрывает более оптимальным `hash-map`. В будущем планируется перейти на сериализацию ключей в единый набор байт и перейти хотя бы на `std::unordered_map`.

- **OrderByOperator** : сортирует результат по выражению-ключу. Сейчас поддерживается сортировка по одному ключу. Используется `std::multimap<key, batch>`.
- **OrderByLimitOperator** : поддерживает сортировку с ограничением количества строк, то есть *top-k* по заданному ключу. Реализация такая же как и в **OrderByOperator**, но в `map` лежит не больше *k* элементов за счет чего асимптотика операций становится лучше. От $O(n \log n)$ к $O(n \log k)$. Также выигрываем по памяти от $O(n)$ к $O(k)$ элементов, хранящихся в `multimap`.
- **LimitOperator** : оставляет первые *k* строк.
- **OffsetOperator** : пропускает первые *k* строк.

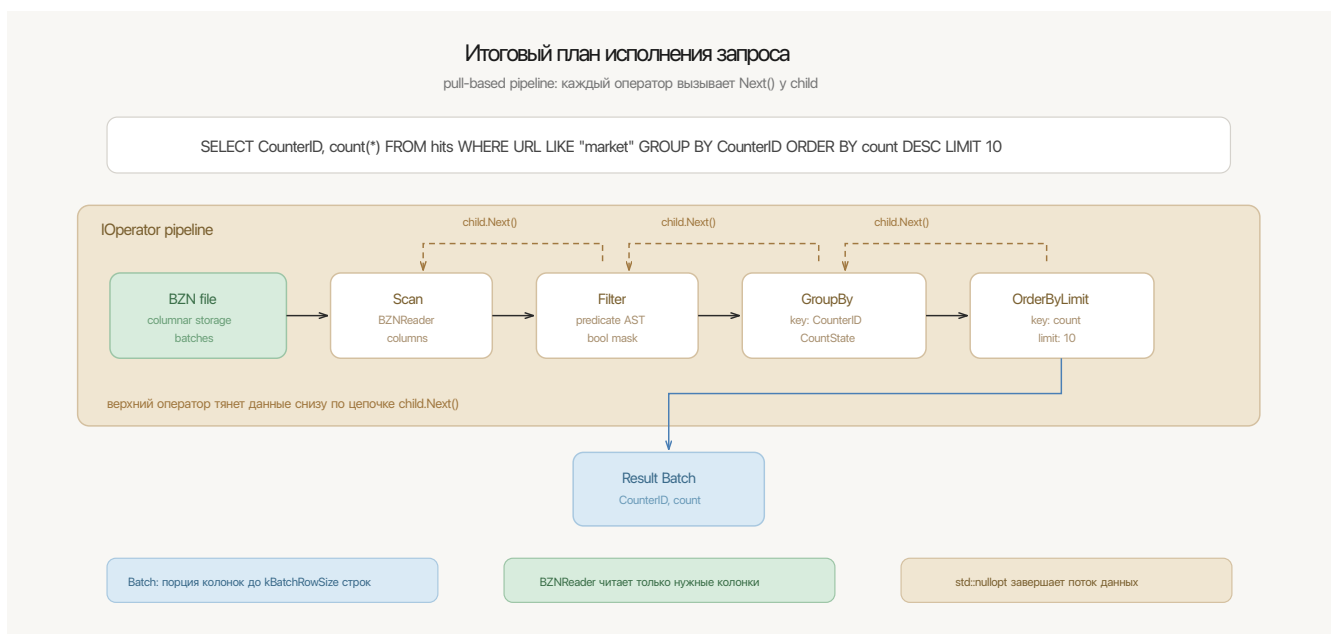


Рис. 3.4: Итоговый план исполнения запроса в pull-based модели.

4 Эксперименты

4.1 Базовая работа конвертации

Экспериментальная часть состоит из перекладывания csv файла в колоночный формат и обратно с различными размерами batch'a (который определяется как количество строк).

Можно обратиться к GitHub [5] и ознакомиться с тестами, на которых проверяется корректность перевода из одного формата в другой. Таблицы из тестов выглядят так:

Таблица 4.1: schema.csv схема из тестов к таблице ниже

a	int64
b	int64
name123	string
d	int64

Таблица 4.2: data.csv тестовая таблица для проверки перекладывания данных

1	2	first	4
5	1	second	2
8	17	third	2

На этих входных данных все корректно перекладывается.

4.2 Время чтения в зависимости от количества колонок

Для оценки эффективности селективного чтения колонок был написан отдельный бенчмарк `bench_read_columns` на базе Google Benchmark. Перед запуском бенчмарка файл `hits_sample.csv` на 999977 строк был один раз переложен в `hits_sample.bzn`. Далее измерялось полное последовательное чтение `bzn` файла с разным количеством выбранных колонок. Между итерациями производилось загрязнение кэша чтением буфера размером 512 MiB.

Бенчмарк запускался в `release` сборке. Для каждого случая было выполнено 5 итераций, в таблице указано среднее реальное время одной итерации.

Бенчмарк запускался на железе : chip Apple M4 Pro, 48gb ram, 1tb APPLE SSD AP1024Z. Версия ОС : macOS Tahoe 26.3

Из таблицы видно, что чтение одной или двух первых колонок выполняется очень быстро, так как это числовые колонки. При чтении четырех и более первых колонок в набор попадает строковая колонка `Title`, которая в текущем формате читается побайтово до разделителя строки. Поэтому время резко возрастает. Селективное чтение колонок работает, но строковые колонки являются главным узким местом текущей реализации.

Таблица 4.3: Время чтения bzn файла в зависимости от количества выбранных колонок

Количество колонок	Строк	Время, мс	Строк/с
1	999977	1.76	$1.14 \cdot 10^8$
2	999977	2.08	$9.59 \cdot 10^7$
4	999977	1164.70	$1.72 \cdot 10^5$
8	999977	1165.65	$1.72 \cdot 10^5$
16	999977	2738.50	$7.30 \cdot 10^4$
32	999977	2811.55	$7.11 \cdot 10^4$
64	999977	3267.75	$6.12 \cdot 10^4$
105	999977	3531.69	$5.66 \cdot 10^4$

Замер чтения всех колонок одного типа из схемы `hits_scheme.csv`: `int16`, `int32`, `int64` и `string`. Колонки типов `DATE` и `TIMESTAMP` в этом замере не учитывались отдельно, достаточно посмотреть на `int32_t` и `int64_t`, в bzn формате они физически представлены одинаково.

Таблица 4.4: Время чтения bzn файла в зависимости от типа выбранных колонок

Тип колонок	Количество колонок	Время, мс	Строк/с
<code>int16</code>	48	17.43	$1.15 \cdot 10^7$
<code>int32</code>	19	13.36	$1.50 \cdot 10^7$
<code>int64</code>	6	8.38	$2.39 \cdot 10^7$
<code>string</code>	28	3534.03	$5.66 \cdot 10^4$

Результаты подтверждают, что числовые типы читаются существенно быстрее строковых: они хранятся как непрерывный массив байт и читаются одним блоком. Строки же сейчас хранятся как последовательность байт с разделителем `'\x1F'` и читаются посимвольно, что приводит к большому числу операций ввода-вывода и обработки каждого символа. Хорошим решением будет в дальнейшем хранить строки как `[длина строки][строка]`, чтобы иметь возможность читать строку блоком.

4.3 Сравнение с DuckDB

Условия сравнений такие же как и выше.

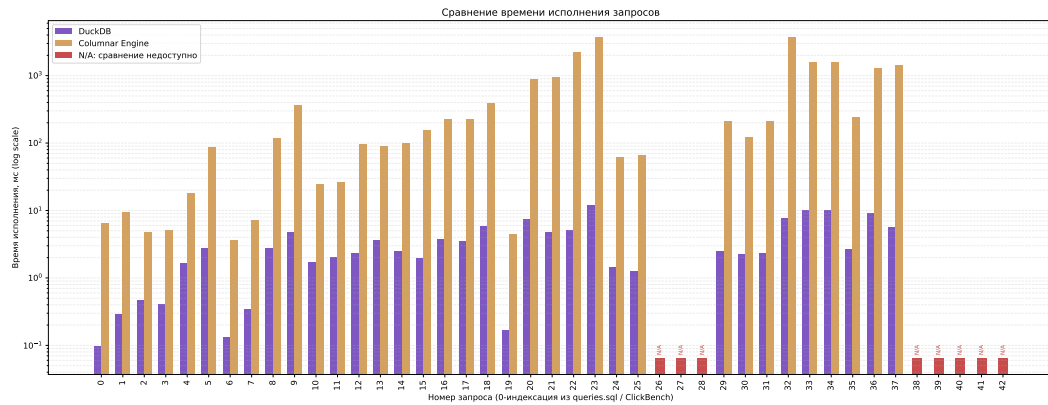


Рис. 4.1: Сравнение с DuckDB.

N/A – сравнение недоступно в реализации отсутствует `ORDER BY` с ключом по больше чем одной колонке и корректная запись пустых батчей, что легко доделывается до готового решения.

Рассмотрим запросы, где `bzn engine` сильно проигрывает DuckDB.

- Запросы 0, 6, 19 очевидно работают хуже, потому что в случае DuckDB они имеют в метаданных `min/max/count` для каждого батча, что позволяет не обновлять состояния и сразу выдавать ответ. Запрос 19 немного выделился на этом фоне, так как это не просто агрегация, а сравнение с `UserId = x`. Благодаря существованию `min/max` можно эффективно пропускать подобные батчи, следовательно, нужно меньше чтений.
- Сильная разница в запросах 20, 21, 22, 23 31 и 32 обусловлена не оптимальной реализацией `groupby` а также чтением колонки строкового типа, это место сильно бьет по производительности. Даже несмотря на эффективный `group by` мы проигрываем по причине неэффективного решения в `bzn` формате.
- Отсутствует использование многопоточности, есть много мест, которые можно оптимизировать используя базовый `thread pool`, например для пересчета агрегаций или `Like` выражения, где можно, к примеру, разбить колонки на две части, считать независимо, после чего просто склеить два ответа вместе.
- Отсутствуют `simd` оптимизации, можно как минимум провести эксперименты с флагами компиляции, влияющих на `simd` и исследовать их влияние на исполнение запросов, как максимум проделать большое количество экспериментов с рукописными `simd` оптимизациями.
- Отсутствует работа с размером батча, чем меньше батч, тем больше нужно проводить

чтений с диска, поиск нужного размера может существенно повлиять на скорость исполнения запросов.

5 План совершенствования проекта

- Переход к более сложной логике в `groupby`, `map` на строках показывает себя очень плохо и тормозит исполнение запросов. Реализация сериализации ключей в набор байт и переход в `hash_map`.
- Улучшение `bzn` формата. Добавить метаданные для колонок, такие как `max`, `min`, количество строк в колонке. Это позволит эффективно пропускать батчи в запросах вида `SELECT * WHERE UserId = 1337`. Благодаря `min/max` такое можно будет быстро пропускать.
- Отсутствует параллелизм и `simd` оптимизации, есть много не исследованных мест, куда это можно добавить и сильно ускорить исполнение.

6 Заключение

Реализован колоночный формат и конверторы из `csv` файла в колоночный формат и обратно. Реализован движок, гибкий к добавлению новой логики со стороны пользователя и способный проходить большую часть запросов ClickBench. Пользователь сам строит план запроса.

Список литературы

- [1] *Apache Parquet information site*. URL: <https://parquet.apache.org>.
- [2] *ClickHouse Benchmark results*. URL: <https://benchmark.clickhouse.com>.
- [3] *ClickHouse information site*. URL: <https://clickhouse.com>.
- [4] *DuckDB information site*. URL: <https://duckdb.org>.
- [5] *GitHub repository project page*. URL: <https://github.com/buziness11/columnar-engine/>.
- [6] *pull model sample*. URL: <https://cs-people.bu.edu/mathan/reading-groups/papers-classics/volcano.pdf>.
- [7] *RFC 4180 спецификация*. URL: <https://www.rfc-editor.org/rfc/rfc4180>.